

# Org Analyser

Step-by-Step Setup Guide

## What is this?

Org Analyser is a tool that automatically looks at all the code repositories (“repos” — think of them as project folders stored on GitHub or GitLab) belonging to a company or team, and produces reports about them: how many pull requests were merged, what kind of work was done, how good the code quality is, and more.

This guide walks through everything needed to get it running on your computer, one step at a time. You do not need to know how to write code — you just need to be able to copy and paste commands into a program called “Terminal.”

**NOTE:** Every gray box in this guide is a command. Copy it exactly (including any words in <angle brackets>, which you replace with your own values), paste it into Terminal, and press Enter.

## Step 1: Install the required tools (one-time setup)

Before running the pipeline, a few free tools need to be installed. This only has to be done once per computer.

### If you are using a Mac

1. Open the “Terminal” app (search for it with Spotlight: press Cmd + Space, type Terminal, press Enter).
2. Install “Homebrew”, a tool that installs other tools. Paste this into Terminal and press Enter:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Follow any on-screen instructions it gives you (it may ask for your Mac password — this is normal).
4. Once Homebrew is installed, install the remaining tools:

```
brew install git scc node
```

### If you are using Windows

1. Install “Chocolatey” by following the instructions at [chocolatey.org/install](https://chocolatey.org/install) (search “install Chocolatey Windows” if needed).
2. Open “PowerShell” (search for it in the Start menu) and run:

```
choco install git nodejs scc -y
```

**NOTE:** If the scc install fails, download it manually from the [scc releases page on GitHub](#), unzip it, and place scc.exe somewhere on your PATH. Ask a technical colleague for help with this step if needed.

## Check everything installed correctly

Paste each line below one at a time. Each should print a version number, not an error:

```
git --version
scc --version
node --version
npx --version
```

## Step 2: Get the Org Analyser code onto your computer

Ask whoever gave you access to this project for the repository URL (or the .zip file you were given). Open Terminal, go to the folder where you want the project to live, and run:

```
git clone <REPOSITORY_URL>
```

This creates a new folder with the project's name. Move into it (replace with the actual folder name that was created):

```
cd <the-folder-that-was-created>
```

**NOTE:** From now on, every command in this guide should be run from inside that folder — the one that contains `cli.py` and `pyproject.toml`.

## Step 3: Set up Python

This tool is written in Python, and it needs its own private, self-contained workspace (called a “virtual environment”) so it doesn't interfere with anything else on your computer. Run these commands one line at a time:

```
python3.12 -m venv .venv
source .venv/bin/activate
python --version
python -m pip install --upgrade pip
python -m pip install -e "[ui]"
```

After running “source .venv/bin/activate”, you should see `(.venv)` appear at the start of your Terminal line. That means it worked. The last command installs everything — the org-analyser command plus every other tool in this project, and the optional Streamlit UI — in one step.

**NOTE:** The “python --version” command must show 3.10 or higher. If it doesn't, try `python3.11` or `python3.10` instead of `python3.12` in the first command above.

**NOTE:** This tool auto-detects the Python inside `.venv` once it is activated, so no extra setup is needed. If you ever need to force a specific interpreter, set `ORG_ANALYSER_PYTHON=$(pwd)/.venv/bin/python` before running it.

**IMPORTANT: If you close Terminal and reopen it later, you will need to redo the “source .venv/bin/activate” line again before running the pipeline. The other install steps only need to happen once.**

## Step 4: Add your access keys ("tokens")

The pipeline needs permission to read repositories and to use OpenAI for its analysis. These permissions come in the form of “tokens” — long secret passwords tied to your account. Never share these or post them publicly.

First, make your own copy of the example config file:

```
cp config.example.yml config.yml
```

Then open the new “config.yml” file in a text editor (TextEdit, VS Code, Notepad, etc.) and fill in the tokens section like this:

```
tokens:
  github-data-token: ghp_your_github_token
  gitlab_token: glpat_your_gitlab_token
  openai_key: sk-your_openai_key
```

## Where to get each key

| Key               | How to get it / when you need it                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| github-data-token | Only needed if analyzing a GitHub org or repo. GitHub.com → Settings → Developer settings → Personal access tokens → generate one with repo read access.                                                                                                                                                                                                                                           |
| gitlab_token      | Only needed if analyzing a GitLab group or project. GitLab → User Settings → Access Tokens → create one with read_api / read_repository scope.                                                                                                                                                                                                                                                     |
| openai_key        | Needed for LLM-based analysis (PR task-profiling and the deepest quality checks). If left blank and OPENAI_API_KEY isn't set in your environment either, those steps fall back to rule-based checks only. Create at <a href="https://platform.openai.com/api-keys">platform.openai.com/api-keys</a> — billed to whoever's OpenAI account you use, so confirm with your team before generating one. |

**NOTE:** *config.yml is deliberately excluded from git (see .gitignore), so your secret keys are never accidentally uploaded anywhere.*

## Step 5: Run the pipeline

Pick the example below that matches what you want to analyze, replace the <placeholders>, and run it. Each run can take anywhere from minutes to several hours depending on how many repositories there are.

### Analyze an entire GitHub organization

```
org-analyser --github-org <ORG_NAME> --workers 10
```

### Analyze a single GitHub repository

```
org-analyser --github-repo <OWNER>/<REPO> --workers 1
```

### Analyze an entire GitLab group

```
org-analyser --gitlab-group <GROUP_NAME> --workers 10
```

### Analyze a single GitLab project

```
org-analyser --gitlab-project <GROUP>/<PROJECT> --workers 1
```

### Analyze a folder of repos already on your computer

```
org-analyser --local-repos-dir ./repos \  
  --repos-manifest repos-manifest.example.json --workers 4
```

**NOTE:** Replace <ORG\_NAME>, <OWNER>/<REPO>, <GROUP\_NAME>, and <GROUP>/<PROJECT> with the real names — remove the angle brackets too. For example:  
`--github-org acme-inc`

**TIP:** Add `--skip-quality-score` to any command above to skip the deepest quality-scoring step, for a faster, simpler run.

## Step 6: Find your results

When the run finishes, look inside the project folder for:

```
outputs/org-analyser-runs/
```

Inside, there will be a new folder named after your org/repo and the date/time of the run. It contains spreadsheets (.xlsx/.csv), data files (.json), logs, and a .zip file with everything bundled together — that zip is the file to share with others.

## Troubleshooting

| Problem                                       | What to try                                                                                                                         |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| The pipeline stops immediately after starting | Check that config.yml exists (copied from config.example.yml) and has the right tokens filled in for the platform you're analyzing. |

| Problem                                                 | What to try                                                                                                                                              |
|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| “SSL: CERTIFICATE_VERIFY_FAILED”                        | Run <code>python -m pip install -e ".[ui]"</code> again to make sure certifi is installed — the tool uses it automatically to supply valid certificates. |
| “command not found: python3.12”                         | Try python3.11 or python3.10 instead, wherever python3.12 appears in Step 3.                                                                             |
| Cloning a repo fails                                    | Double check your github-data-token or gitlab_token (in config.yml) has access to that repository, and that you typed the org/repo name correctly.       |
| Warnings about the code profiler                        | Make sure git, scc, and node were all installed successfully in Step 1 (re-run the version-check commands).                                              |
| Some repos fail but others succeed                      | This is expected on large organizations. The run continues automatically — check manifest.json in the output folder to see what succeeded.               |
| Terminal shows (env) or another name instead of (.venv) | Run "deactivate", then redo the "source .venv/bin/activate" command from Step 3.                                                                         |
| “one target required” error                             | Pass one of --github-org, --github-repo, --gitlab-group, --gitlab-project, or --local-repos-dir, or set a default target inside config.yml.              |

## Glossary (plain-language definitions)

| Term                       | Meaning                                                                                                                                                              |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Terminal                   | A text-based program on your computer where you type commands instead of clicking buttons.                                                                           |
| Repo (repository)          | A project's folder of code, stored online on GitHub or GitLab.                                                                                                       |
| Org (organization)         | A group of repos owned by one company or team on GitHub/GitLab.                                                                                                      |
| Clone                      | Making a copy of an online repo onto your own computer.                                                                                                              |
| Token                      | A secret password that proves you have permission to access something, like a repo or an AI service. Stored locally in config.yml, never shared or committed to git. |
| Virtual environment (venv) | A private, isolated space for Python so this tool's software doesn't conflict with anything else installed on your computer.                                         |
| Pull Request (PR)          | A proposed change to a codebase that a developer submits for review before it's added to the project.                                                                |